Reviewer Pack — Free Cumulant κ₄ of Disjointness Spectrum Lower-Bounds R(DIS…

Entry c30c16c159e9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 21:02:33 UTC

Free Cumulant κ₄ of Disjointness Spectrum Lower-Bounds R(DISJ)

Entry ID: `c30c16c159e9` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-07 21:02:33 UTC

1. Conjecture

Field A (mathematical branch): Free probability (Voiculescu free cumulants of empirical singular-value distributions) **Field B** (complexity object): SoS degree-2 moment matrix view of randomized communication complexity of DISJOINTNESS

Statement:

> For k ≥ 2 let Π_k ∈ {−1,+1}^{2k×2^k} be the sign-disjointness matrix Π_k[x,y] = (−1)^{[x∧y ≠ 0]}, and let μ_k be the empirical distribution of the squared singular values of Π_k/√(N·‖Π_k‖_F²/N) (so m₁(μ_k)=1, N=2^k). Define the fourth free cumulant κ₄(μ) := m₄(μ) − 2·m₂(μ)². We conjecture (i) κ₄(μ_k) ≥ c·k for an absolute constant c > 0, while (ii) for an N×N uniformly random ±1 sign matrix R, E[κ₄(μ_R)] = O(1/N); equivalently, the κ₄ of the SoS-degree-2 moment matrix of any sign matrix M furnishes a lower bound R^pub(M) ≥ Ω(κ₄(μ_M)) on randomized public-coin communication complexity, and DISJ saturates this bound to Ω(k).

Rationale (proposer's reasoning):

> Random sign matrices have semicircular limiting spectra whose free cumulants vanish above order 2, so κ₄ detects deviation from freeness — exactly the structured 'tensor-power' rigidity that makes DISJ hard. Because Π_k = M^{⊗k} for the 2×2 sign block M, its squared spectrum is the k-fold classical convolution of a non-semicircular law, forcing κ₄ to grow linearly in k under the multiplicativity of moments. Free cumulants are read directly off the SoS degree-2 pseudoexpectation moment matrix, giving a clean SoS-hierarchy invariant that is unrelativizing (depends on actual matrix entries) and avoids algebrization (no oracle algebraic extension can fake non-semicircular spectra of explicit tensor-power matrices).

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `64ae81e2a5341a8b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across k∈{2,3,4,5,6}, $\kappa_4(\Pi_k)$ must be strictly monotonically increasing with OLS linear-fit slope ≥0.1, AND at each k must exceed mean+3·std of κ_4 over 30 random ±1 sign matrices of size $2^{k\times 2}$k (equivalently lie strictly above the empirical 97.5% quantile).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `free cumulants singular value distribution disjointness communication complexity` - `spectral lower bound randomized communication complexity moment matrix sum of squares` - `free probability empirical spectral distribution sign matrix communication complexity`

Top relevant hits considered: - [http://arxiv.org/abs/2207.14810v3] Simplifying a classical-quantum algorithm interpolation with quantum singular value transformations - [http://arxiv.org/abs/2303.0071 Full Eigenstate Thermalization via Free Cumulants in Quantum Lattice Systems - [http://arxiv.org/abs/2603.194 Communication Complexity of Disjointness under Product Distributions - [http://arxiv.org/abs/1608.06580v2] Communication complexity of approximate Nash equilibria - [http://arxiv.org/abs/1210.1461v1] A Scalable CUR Matrix Decomposition Algorithm: Lower Time Complexity and Tighter Bound - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size - [http://arxiv.org/abs/1204.6227v2] On the spectral distribution of the free Jacobi process - [http://arxiv.org/abs/1204.6522v3] Formal Groups, Witt vectors and Free Probability

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, n):
```

```
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def frobenius_norm(A):
    n = len(A)
    norm = 0
    for i in range(n):
        for j in range(n):
            norm += A[i][j]**2
    return math.sqrt(norm)

def sign_disjointness_matrix(k):
    n = 2**k
    M = [[(-1)**(i & j != 0) for j in range(n)] for i in range(n)]
    return M

def singular_values(A):
    A_norm = frobenius_norm(A)
    U, _, Vt = gaussian_elimination(matrix_multiply(A, A))
    return [math.sqrt(U[i][i] * Vt[i][i]) / A_norm for i in range(len(A))]

def free_cumulant_4(mu):
    m2 = sum(x**2 for x in mu) / len(mu)
    m4 = sum(x**4 for x in mu) / len(mu)
    return m4 - 2 * m2**2

k_values = [2, 3, 4, 5, 6]
results = []

for k in k_values:
    Pi_k = sign_disjointness_matrix(k)
    sv = singular_values(Pi_k)
    mu_k = [x**2 / sum(sv) for x in sv]

    kappa_4 = free_cumulant_4(mu_k)
    results.append({"k": k, "kappa_4": kappa_4})

mean_kappa_4 = sum(result["kappa_4"] for result in results) / len(results)
std_kappa_4 = math.sqrt(sum((result["kappa_4"] - mean_kappa_4)**2 for result in results) / len

conjecture_holds = all(result["kappa_4"] > mean_kappa_4 + 3 * std_kappa_4 for result in result
counterexample = "" if conjecture_holds else "Non-monotonic k→κ₄(Π_k) profile or κ₄(Π_k) insid

return {
    "metric_name": "kappa_4",
```

```
        "metric_value": mean_kappa_4,
        "instances_tested": len(k_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_kappa_4 = sum(r["metric_value"] for r in results) / len(results)
    std_kappa_4 = math.sqrt(sum((r["metric_value"] - mean_kappa_4)**2 for r in results) / len(resu
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_kappa_4} std={std_kappa_4} support_fraction={support_
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Non-monotonic k→κ₄(Π_k) profile or κ₄(Π_k) insi
    else:
        print(f"RESULT: INCONCLUSIVE No seeds tested")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_76f8276b.py", line 95, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_76f8276b.py", line 69, in run_trial
    sv = singular_values(Pi_k)
         ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_76f8276b.py", line 56, in singular_valu
    U, _, Vt = gaussian_elimination(matrix_multiply(A, A))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_76f8276b.py", line 27, in gaussian_elim
    factor = A[j][i] / A[i][i]
             ~~~~~~~~^~~~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with ZeroDivisionError in a hand-rolled gaussian_elimination routine before any κ_4 values were computed, so neither the support nor falsification conditions can be evaluated. | next: Replace the custom SVD with numpy.linalg.svd (or eigvalsh on AA^T) to robustly compute singular values of Π_k and random sign matrices, then re-run the pre-registered criterion across k∈{2..6}.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	109833
2	preregistration	claude_max	opus	0	0	7931
3	novelty	claude_max	opus	0	0	3613
4	novelty	claude_max	opus	0	0	9231
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	76195
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	117444
7	judge	claude_max	opus	0	0	4417

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 328662 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/c30c16c159e9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c30c16c159e9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c30c16c159e9.tar.gz` (if generated)